

CHRIST (Deemed to be University), Bengaluru
Department of Computer Science

Sprint I: Design and Development Document

Course: CSC481-5, Computer Science Project
Semester V

ThinkStack

Offline SLM-Based Research Assistant

Submitted by

Sl. No.	Student Name	Register No.
1.	Aditya Mehta	2440204
2.	Jitvan Chadha	2440222
3.	Ritesh K R	2440233

Guide: Dr. New Begin
Class: 5BSC CS
Repository: github.com/get-thinkstack/ThinkStack

July 2026

Contents

1	Sprint Overview	2
1.1	Period and scope	2
1.2	Sprint goal statement	2
2	Requirements Addressed	2
2.1	Functional requirements	2
2.2	Non-functional requirements	2
3	System Design	3
3.1	Architectural style	3
3.2	Ingestion pipeline design	3
3.3	Retrieval design	3
3.4	Storage design	4
3.5	Encryption design	4
3.6	Paper writer design	4
4	Implementation	4
4.1	Technology selected	4
4.2	Model selection	5
4.3	Work completed	5
5	Testing Performed	5
6	Sprint I Outcome	5
6.1	Retrospective	5

1 Sprint Overview

1.1 Period and scope

Sprint I ran from 5 June 2026 to 24 June 2026. Its objective was to establish a working research assistant in which every core capability functioned end to end with local inference: ingesting research papers, retrieving from them, summarising them, conversing about them, protecting them, and drafting LaTeX.

Attribute	Value
Duration	5 June 2026 to 24 June 2026
Active development days	11
Commits	29
Deliverable	a functional application running from source

The team of three worked jointly across the sprint, with responsibilities rotating between backend services, interface work, and the desktop shell. Work was integrated through pull requests into a shared branch.

1.2 Sprint goal statement

Goal. A user should be able to add a PDF to the application, ask a question about it, receive an answer drawn from that document rather than from the model’s memory, and draft a LaTeX section with assistance, with all computation performed on the local machine and no network dependency.

2 Requirements Addressed

2.1 Functional requirements

ID	Requirement	Realisation in Sprint I
FR-1	Ingest PDF documents	extraction, metadata recovery, chunking, embedding, storage
FR-2	Search the collection	dense semantic retrieval combined with keyword retrieval
FR-3	Summarise a document	structured summarisation through the local model
FR-4	Extract key claims	claim extraction returning structured output
FR-5	Cluster themes	thematic grouping across the corpus
FR-6	Identify research gaps	gap analysis with suggestion generation
FR-7	Converse about papers	retrieval-grounded chat over selected documents
FR-8	Protect sensitive papers	password-based encryption with authenticated ciphertext
FR-9	Draft LaTeX	AI-assisted editor with local compilation to PDF

2.2 Non-functional requirements

- **Privacy.** No document, prompt, or derived artefact may leave the machine. This is the defining constraint of the product and it eliminated every hosted-inference design.
- **Offline operation.** The application must function with the network disconnected.
- **Commodity hardware.** The target is an ordinary student laptop, assumed to have 8 to 16 GB of memory and no usable GPU.
- **No running cost.** No API keys and no subscription.

3 System Design

3.1 Architectural style

A layered architecture was adopted, with a REST interface between the user interface and the domain logic.

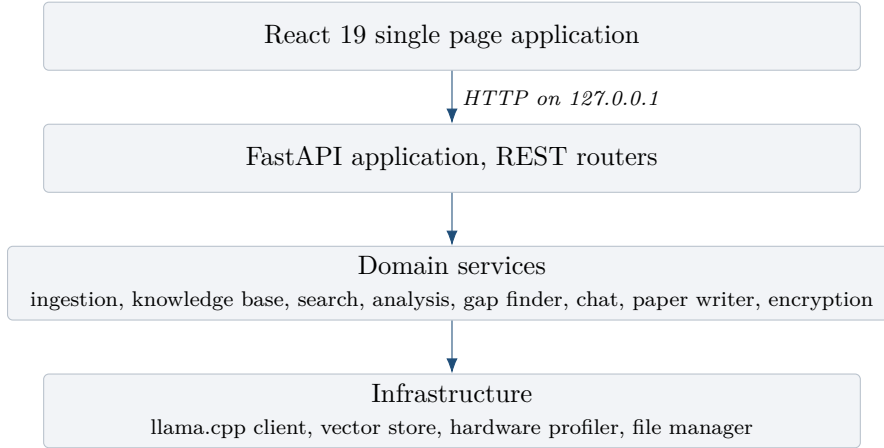


Figure 1: Layered architecture established in Sprint I.

Design rationale. A local HTTP boundary was chosen over embedding the Python runtime directly into the desktop shell. Three properties followed from that decision: the same backend serves both a desktop window and an ordinary browser without duplicated code; a failure inside the model runtime terminates a supervised child process rather than the whole application; and the interface between layers is inspectable with ordinary HTTP tooling, which later made automated validation of the packaged product possible.

3.2 Ingestion pipeline design



Figure 2: Document ingestion. Chunks are 512 tokens with 50 tokens of overlap.

Chunking strategy. Text is divided by a descending hierarchy of separators: paragraph breaks first, then line breaks, then sentence boundaries, then words. This preserves semantic units, so a retrieved chunk is more likely to be independently meaningful. Overlap of 50 tokens ensures that a statement spanning a boundary is not lost to both neighbours.

Metadata recovery. Regular expressions extract title, authors, abstract and year. The local model is invoked only when that leaves the title or abstract empty, so an expensive inference call is avoided for well-formed papers.

3.3 Retrieval design

Two retrieval methods are computed and fused rather than one being chosen:

- **Dense retrieval** embeds the query into the same 384-dimensional space as the chunks and ranks by cosine similarity. This finds paraphrase, where the query and the document share meaning but no words.

- **Keyword retrieval** scores by BM25. This finds exact technical terms, identifiers and notation, where dense retrieval is weak.
- **Reciprocal rank fusion** combines the two ranked lists, rewarding documents that both methods rank highly without requiring the two score scales to be comparable.

3.4 Storage design

A file-based vector store using NumPy cosine similarity was chosen over an embedded database. The reasoning was the governing constraint: a database engine is another component that must install correctly on a machine the developers cannot inspect. The accepted cost is that the keyword index is rebuilt per query, which is acceptable for a personal library and is recorded as a known limitation.

3.5 Encryption design

Concern	Mechanism
Key derivation	Argon2id, memory-hard, resistant to hardware brute force
Confidentiality and integrity	AES-256-GCM, authenticated encryption
Format	versioned envelope, so the scheme can change without data loss

Authenticated encryption was chosen deliberately over an unauthenticated mode: tampering with a stored paper is detected at decryption rather than producing plausible but corrupted output.

3.6 Paper writer design

The paper writer accepts a natural language description and returns compilable LaTeX produced by the local model. The compiler was designed to be forgiving, because a language model of this size produces markup that is frequently almost correct:

- missing package declarations are inserted automatically;
- a bare fragment is wrapped into a complete document;
- compilation continues past recoverable errors, and the existence of a PDF is treated as success with errors surfaced as warnings;
- when a pass produces no PDF, the failing environment is progressively neutralised and compilation retried, so a single malformed figure does not cost the author the entire document.

4 Implementation

4.1 Technology selected

Concern	Choice	Reason
Backend	FastAPI, Uvicorn	typed request models, automatic API schema
Interface	React 19, Vite	component model suited to distinct feature panes
Desktop shell	Tauri 2, Rust	small binary, uses the system webview
Inference	llama.cpp, GGUF	runs quantised models on CPU
Embeddings	all-MiniLM-L6-v2	384 dimensions, small enough to bundle
Keyword search	rank_bm25	no service to install
PDF handling	PyMuPDF, pdfplumber	PyMuPDF for speed, pdfplumber as fallback
Encryption	argon2-cffi, cryptography	standard, audited implementations

4.2 Model selection

Qwen2.5 instruction models were selected at 0.5 and 1.5 billion parameters, quantised to four bits in the Q4_K_M scheme. The 0.5B model is used for conversational tasks where latency is visible to the user, and the 1.5B model for tasks that must emit structured output. Quantisation is what makes CPU execution viable within the memory of an ordinary laptop.

4.3 Work completed

Date	Outcome
5 to 10 June	repository established, initial documentation, first working demonstration
11 to 16 June	interface iteration, project naming corrected in documentation
17 June	project renamed from ScholarLens to ThinkStack; backend audit and correction; architecture decision log and feature reference established
17 June	chat assistant verified against the local runtime
19 to 20 June	desktop application initialised with Tauri; first DevOps scripts
20 to 21 June	encryption and decryption implemented; interface refinements; edge cases in the encryption flow corrected
23 June	LaTeX editor and auto-healing compiler implemented
24 June	interface unified with the LaTeX editor; desktop packaging configured; security design documented

5 Testing Performed

Testing in Sprint I was manual and feature-directed. Each capability was exercised through the interface against real research papers: a document was ingested, searched, summarised, discussed through chat, encrypted and decrypted, and a LaTeX section was drafted and compiled.

No automated test suite existed at the end of Sprint I. This is stated plainly because it became the central weakness addressed in Sprint II.

6 Sprint I Outcome

Objective	Achieved
Ingest and index PDF documents	Yes
Hybrid semantic and keyword retrieval	Yes
Summarisation, claim extraction, thematic clustering	Yes
Research gap identification	Yes
Retrieval-grounded chat	Yes
Local document encryption	Yes
AI-assisted LaTeX editor with compilation	Yes
Desktop shell around the application	Yes
Runs with the network disconnected	Yes
Installable by a user on an unprepared machine	No
Automated regression testing	No

6.1 Retrospective

What succeeded. Every technically demanding component was completed within the sprint. Local inference with acceptable latency, hybrid retrieval, authenticated encryption and a fault-tolerant LaTeX compiler all worked. Recording architectural decisions as they were taken, beginning on 17 June, produced a written rationale that proved valuable in the following sprint.

What was deferred. The application ran from a source checkout on machines that already had Python, a model runtime and a LaTeX distribution installed. Nothing had been packaged, distributed, or executed anywhere else. The distinction between software that works and software that has been shown to work on another machine was not yet appreciated, and defined the entire second sprint.

Carried into Sprint II.

1. Package the application into installers for Linux, macOS and Windows.
2. Remove every assumption that the user's machine resembles a developer's.
3. Introduce automated testing.
4. Establish a repeatable release process.